Practical no:3

Aim: To implement a Deep Q-Network (DQN) using a deep learning library (TensorFlow/PyTorch) and train the model to solve a reinforcement learning environment such as Cart Pole or Mountain Car.

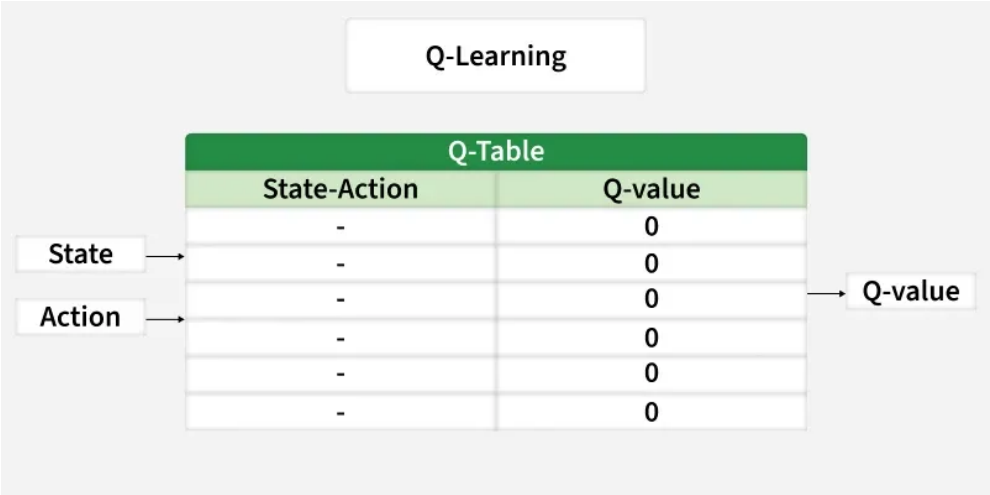
Objectives:

- To understand the fundamentals of Reinforcement Learning (RL).
- To study Q-Learning and its limitations.
- To implement Deep Q-Network using neural networks.
- To train an agent to take optimal actions in an environment.
- To analyze the performance of the trained agent.

Theory:

Deep Q-Learning is a method that uses deep learning to help machines make decisions in complicated situations. It's especially useful in environments where the number of possible situations called states is very large like in video games or robotics.

Before understanding Deep Q-Learning it's important to understand the main concept of Q-Learning. It is a model-free method that learns an optimal policy by estimating the Q-value function which tells how good it is to take a certain action in a certain situation. The goal is to find a plan that gives the highest total reward over time.



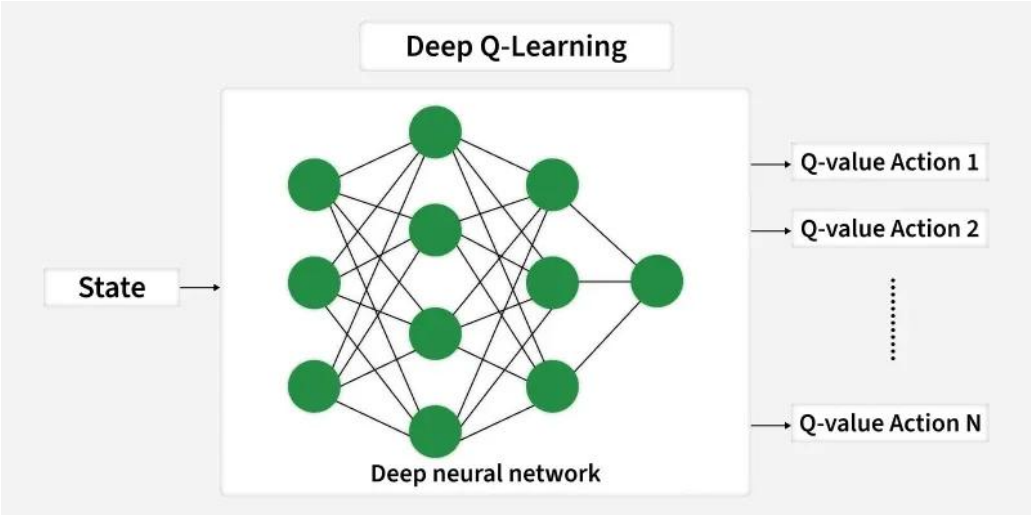
Q-Learning works well for small problems but struggles with complex ones like images or many possible situations. Deep Q-Learning solves this by using a neural network to estimate values instead of a big table.

Challenges Addressed by Deep Q-Learning:

- **High-Dimensional State Spaces:** Traditional Q-Learning uses a table to store values but this becomes impossible when there are too many situations. Neural networks can understand and work with many different situations at once so they are better for complex problems.
- **Continuous Input Data:** Real-world problems often have continuous data like video images. Neural networks are good at handling this kind of information.

- **Scalability:** Deep learning helps Q-Learning grow and handle bigger, harder tasks that regular Q-Learning couldn't solve before.

Architecture of Deep Q-Networks
A DQN consists of the following components:



a) Neural Network

- Input: State
- Output: Q-values for all actions

b) Experience Replay

- Stores past experiences (state, action, reward, next state)
- Breaks correlation between samples
- Improves learning stability

c) Target Network

- Separate network for stable Q-value updates
- Updated periodically

DQN Algorithm:

1. Initialize replay memory
2. Initialize Q-network with random weights
3. For each episode:
    - Observe state
    - Select action (ε-greedy policy)
    - Execute action
    - Store experience in memory
    - Sample random batch
    - Train neural network
    - Update target network periodically

Applications

Deep Q-Learning is used in many areas such as:

- **Atari Games:** It can learn to play old video games very well even better than humans by looking at the screen pixels.
- **Robotics:** It helps robots to learn how to pick objects, move around and do tasks with their hands.
- **Self-Driving Cars:** It helps cars to make decisions like changing lanes and avoiding obstacles safely.
- **Finance:** It is used to find the best ways to trade stocks, manage money and reduce risks.
- **Healthcare:** It helps with planning treatments, discovering new medicines and personalizing care for patients.

Conclusion:

The Deep Q-Network (DQN) was successfully implemented, enabling the agent to learn optimal actions through interaction with the environment. This experiment demonstrates the effectiveness of combining deep learning with reinforcement learning for solving complex decision-making problems.